SIMPLY FPU

by **Raymond Filiatreault**

Chap. 10
Trigonometric instructions

The FPU instructions covered in this chapter perform trigonometric operations with the value in the TOP data register ST(0), or with both ST(0) and ST(1).

The trigonometric instructions covered in this document are (in alphabetical order):

FSIN (Sine of the angle value in ST(0))

```
Syntax:    fsin (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Underflow, Precision
```

This instruction computes the sine of the source angle value in ST(0) and overwrites the content of ST(0) with the result. The angle must be expressed in radians and be within the -2^{63} to $+2^{63}$ range.

If the source angle value is outside the acceptable range (but not INFINITY), the C2 flag of the Status Word is set to 1 and the content of ST(0) remains unchanged, no exception being detected. *(The source value can be reduced to within the acceptable range with the FPREM instruction using a divisor of 2π.)*

An **I**nvalid operation exception is detected if the TOP data register ST(0) is empty, or is a NAN, or has a value of INFINITY, setting the related flag in the Status Word. The content of ST(0) would be overwritten with the INDEFINITE value.

A **S**tack **F**ault exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when the content of ST(0) is a denormalized number or the result is a denormalized number, setting the related flag in the Status Word.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **U**nderflow exception will be detected if the result exceeds the range limit of REAL10 numbers, setting the related flag in the Status Word.

The relation between degrees and radians is as follows:

180 degrees = π radians

To compute the sine of an angle expressed in degrees in ST(0), the following code could be used to first convert the value in ST(0) from degrees to radians and check for errors before the FSIN instruction.

```
    fclex         ;clear all previous exceptions
    pushd 180     ;store the integer value of 180 on the stack
    fidiv dword ptr[esp]   ;divide the angle in degrees by 180
                  ;-> ST(0)=angle in degrees/180
    fldpi         ;load the hard-coded value of π
                  ;-> ST(0)=π, ST(1)=angle in degrees/180
    fmul          ;-> ST(0)=angle in degrees*π/180, => angle in radians
    fsin          ;compute the sine of the angle
                  ;-> ST(0)=sin(angle) if no error
    fstsw [esp]   ;store the Status Word on the stack overwriting the 180
    fwait         ;to insure the last instruction is completed
    pop   eax     ;get the Status Word in AX (which also cleans the stack)
    shr   al,1    ;transfer the Invalid op flag (bit0 of AL) to the CF flag
    jnc   @F      ;jump if no invalid operation detected

    ........     ;insert code to handle an invalid operation
                  ;-> ST(0)=INDEFINITE (angle has been trashed)

@@:
    sahf          ;transfer AH to the CPU flag register
    jpo   @F      ;jump if PF=C2=0 meaning angle value is in acceptable range

    ........     ;insert code to handle angle outside acceptable range
                  ;-> ST(0)=angle in radians unchanged

@@:               ;-> ST(0)=sin(angle) -- no error
```

FCOS (Cosine of the angle value in ST(0))

```
Syntax:    fcos (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Underflow, Precision
```

This instruction computes the cosine of the source angle value in ST(0) and overwrites the content of ST(0) with the result. The angle must be expressed in radians and be within the -2^{63} to $+2^{63}$ range.

If the source angle value is outside the acceptable range (but not INFINITY), the C2 flag of the Status Word is set to 1 and the content of ST(0) remains unchanged, no exception being detected. *(The source value can be reduced to within the acceptable range with the FPREM instruction using a divisor of 2π.)*

An **I**nvalid operation exception is detected if the TOP data register ST(0) is empty, or is a NAN, or has a value of INFINITY, setting the related flag in the Status Word. The content of ST(0) would be overwritten with the INDEFINITE value.

A **S**tack **F**ault exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when the content of ST(0) is a denormalized number or the result is a denormalized number, setting the related flag in the Status Word.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **U**nderflow exception will be detected if the result exceeds the range limit of REAL10 numbers, setting the related flag in the Status Word.

The relation between degrees and radians is as follows:

180 degrees = π radians

To compute the cosine of an angle expressed in degrees in ST(0), the following code could be used to first convert the value in ST(0) from degrees to radians before the FCOS instruction. The code assumes that ST(0) contains a valid value within the acceptable range, such that no error checking is necessary.

```
pushd 180    ;store the integer value of 180 on the stack
fidiv dword ptr[esp]   ;divide the angle in degrees by 180
             ;-> ST(0)=angle in degrees/180
fldpi        ;load the hard-coded value of π
             ;-> ST(0)=π, ST(1)=angle in degrees/180
fmul         ;-> ST(0)=angle in degrees*π/180, => angle in radians
add   esp,4  ;adjust the stack pointer to clean-up the pushed 180
             ;pop reg32 would also clean the stack but trash a register
fcos         ;compute the cosine of the angle
             ;-> ST(0)=cos(angle)
```

FSINCOS (Sine and cosine of the angle value in ST(0))

```
Syntax:    fsincos (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Underflow, Precision
```

This instruction computes the sine and the cosine of the source angle value in ST(0). The sine value replaces the content of ST(0), the TOP register field of the Status Word is decremented, and the cosine value is inserted into the new ST(0). The angle must be expressed in radians and be within the -2^{63} to $+2^{63}$ range. (This instruction is faster than computing the sine and cosine separately with the FSIN and FCOS instructions.)

If the source angle value is outside the acceptable range (but not INFINITY), the C2 flag of the Status Word is set to 1 and the content of all data registers remains unchanged; the TOP register field of the Status Word is not modified and no exception is detected. *(The source value can be reduced to within the acceptable range with the FPREM instruction using a divisor of 2π.)*

An **I**nvalid operation exception is detected if the TOP data register ST(0) is empty, or is a NAN, or has a value of INFINITY, or the ST(7) data register is not empty, setting the related flag in the Status Word. The content of ST(0) would be overwritten with the INDEFINITE value, the TOP register field of the Status Word would be decremented, and the content of the new ST(0) would also be overwritten with the INDEFINITE value.

A **S**tack **F**ault exception is also detected if ST(0) is empty or if ST(7) is not empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when the content of ST(0) is a denormalized number or a result is a denormalized number, setting the related flag in the Status Word.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **U**nderflow exception will be detected if the result exceeds the range limit of REAL10 numbers, setting the related flag in the Status Word.

The relation between degrees and radians is as follows:

180 degrees = π radians

To compute the sine and cosine of an angle expressed in degrees in ST(0), the following code could be used to first convert the value in ST(0) from degrees to radians and verify that it is within the acceptable range before the FSINCOS instruction. This code does not differentiate invalid operations from a value simply out of range.

```
pushd 180    ;store the integer value of 180 on the stack
fidiv dword ptr[esp]   ;divide the angle in degrees by 180
             ;-> ST(0)=angle in degrees/180
fldpi        ;load the hard-coded value of π
             ;-> ST(0)=π, ST(1)=angle in degrees/180
fmul         ;-> ST(0)=angle in degrees*π/180, => angle in radians
```

```
    fst   dword ptr[esp] ;use the already reserved space on the stack
                ;to get a REAL4 version of the value in ST(0)
    fwait       ;to insure the last instruction is completed
    pop   eax   ;retrieve the REAL4 in EAX (which also cleans the stack)
    rol   eax,9 ;the 8 biased exponent bits => to the lower 8 bits of EAX
                ;(the sign bit would become the least significant bit of AH)
    cmp   al,127+64    ;the exponent bias for the REAL4 format is 127
    jb    @F    ;the value in ST(0) would be within range if AL < 127+64

    .......     ;insert code to handle the unacceptable source value
                ;if AL=255, source value was >2^127 or was NAN

@@:
    fsincos     ;compute both the sine and cosine of the angle
                ;-> ST(0)=cos(angle), ST(1)=sin(angle)
;all other values in data registers would be in the ST(i+1) register
```

FPTAN (Partial tangent of the angle value in ST(0))

```
Syntax:    fptan (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Underflow, Precision
```

This instruction computes the tangent of the source angle value in ST(0). The tangent value replaces the content of ST(0), the TOP register field of the Status Word is decremented, and a value of 1.0 is inserted into the new ST(0). The angle must be expressed in radians and be within the -2^{63} to $+2^{63}$ range.

> The extra value of 1.0 is primarily for compatibility with the early co-processors prior to the 387. The FSIN and FCOS instructions were not then available and had to be computed from the tangent value (and the acceptable range for the angle was only 0 to $+\pi/4$). It is more a nuisance than a feature with the more modern FPUs, requiring the need for two registers instead of one and an extra instruction to discard it.

If the source angle value is outside the acceptable range (but not INFINITY), the C2 flag of the Status Word is set to 1 and the content of all data registers remains unchanged; the TOP register field of the Status Word is not modified and no exception is detected. *(The source value can be reduced to within the acceptable range with the FPREM instruction using a divisor of 2π.)*

An **I**nvalid operation exception is detected if the TOP data register ST(0) is empty, or is a NAN, or has a value of INFINITY, or the ST(7) data register is not empty, setting the related flag in the Status Word. The content of ST(0) would be overwritten with the INDEFINITE value, the TOP register field of the Status Word would be decremented, and the content of the new ST(0) would also be overwritten with the INDEFINITE value.

A **S**tack **F**ault exception is also detected if ST(0) is empty or if ST(7) is not empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when the content of ST(0) is a denormalized number or a result is a denormalized number, setting the related flag in the Status Word.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **U**nderflow exception will be detected if the result exceeds the range limit of REAL10 numbers, setting the related flag in the Status Word.

Although the tangent would have a value of infinity when the angle would be exactly equal to $\pi/2$, the maximum value returned by the FPTAN instruction will only be approximately $3.69*10^{19}$. This properly reflects the limit of precision of the FPU.

Because an additional value is loaded to a data register unless this instruction is executed on a value outside the acceptable range, all other values in data registers would then be in the ST(i+1) register.

The tangent of an angle expressed in radians and located in ST(0) is obtained as follows.

```
                ;ST(0)=angle in radians, ST(1)=xx
fptan           ;ST(0)=1.0, ST(1)=tan(angle), ST(2)=xx
fstp st         ;this pops the TOP register
                ;ST(0)=tan(angle), ST(1)=xx
```

FPATAN (Partial arctangent of the ratio ST(1)/ST(0))

```
Syntax:    fpatan (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Underflow, Precision
```

This instruction computes the arctangent of the ST(1)/ST(0) ratio, overwrites the content of ST(1) with the angle value (in radians) and then POPs the TOP data register. The result will be a value in the range of -π to +π (-180 to +180 if converted to degrees) depending on the signs of the contents of ST(0) and ST(1).

The following tabulation gives the resulting values (or range of values) converted to degrees based on the contents of ST(0) and ST(1). "F" means a finite value between 0 and INFINITY.

ST(1) ST(0)	+∞	+F	+0	-0	-F	-∞
+∞	+45°	+0°	+0°	-0°	-0°	-45°
+F	+90°	+0° to +90°	+0°	-0°	-90° to -0°	-90°
+0	+90°	+90°	+0°	-0°	-90°	-90°
-0	+90°	+90°	+180°	-180°	-90°	-90°
-F	+90°	+90° to +180°	+180°	-180°	-180° to -90°	-90°
-∞	+135°	+180°	+180°	-180°	-180°	-135°

An **I**nvalid operation exception is detected if either ST(0) or ST(1) is empty or is a NAN, setting the related flag in the Status Word. The TOP data register would be POPed and the content of ST(0) (formerly ST(1)) would be overwritten with the INDEFINITE value.

A **S**tack **F**ault exception is also detected if either ST(0) or ST(1) is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when the content of either ST(0) or ST(1) is a denormalized number or the result is a denormalized number, setting the related flag in the Status Word.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **U**nderflow exception will be detected if the result exceeds the range limit of REAL10 numbers, setting the related flag in the Status Word.

Because the TOP data register is POPed with this instruction, all other values in data registers now would be in the ST(i-1) register.

The arctangent can be computed in two different manners. One of them is when the actual value of the tangent is in ST(0), either from prior computations or loaded from memory. By loading a value of +1.0 followed by the FPATAN instruction, the arctangent result would be in the **-90° to 0°** or **0° to +90°** ranges depending on the sign of the tangent value. Loading a value of -1.0 would give results in the **+90° to +180°**

or **-180° to -90°** ranges. Some prior knowledge of the required ranges will be necessary in order to load the 1.0 with the proper sign. Typical code would be:

```
                ;ST(0)=value of tangent, ST(1)=xxx
fld1            ;ST(0)=+1.0, ST(1)=value of tangent, ST(2)=xxx
fchs            ;change to -1.0 for angle in the +90° to 180° to -90° ranges
                ;ST(0)=-1.0, ST(1)=value of tangent, ST(2)=xxx
fpatan          ;ST(0)=angle in radians, ST(1)=xxx
pushd 180       ;for converting to degrees
fimul dword ptr[esp] ;ST(0)=180*angle in radians, ST(1)=xxx
fldpi           ;ST(0)=π, ST(1)=180*angle in radians, ST(2)=xxx
fdiv            ;ST(0)=180/π*angle in radians=angle in degrees, ST(1)=xxx
add   esp,4     ;adjust the stack pointer to clean-up the pushed 180
                ;pop reg32 would also clean the stack but trash a register
```

The other manner is when the signed dimensions of the sides of a right triangle (or Cartesian coordinates) are available. The dimension of the side opposite the angle is loaded first followed by the dimension of the side adjacent to the angle. The result will then cover the entire circle.

FPU instructions are not available to directly compute the **arcsine** and the **arccosine**. The FPATAN instruction must be used for that purpose based on the usual trigonometric equivalents:

$$\tan(x) = \sin(x)/\cos(x)$$

and

$$\sin^2(x) + \cos^2(x) = 1$$

Typical code to compute arcsin[sin(x)] from the sine value in ST(0) would be as follows.

```
                ;ST(0)=sin(x), ST(1)=zzz
fld   st        ;ST(0)=sin(x), ST(1)=sin(x), ST(2)=zzz
fmul  st,st     ;ST(0)=sin²(x), ST(1)=sin(x), ST(2)=zzz
fld1            ;ST(0)=1.0, ST(1)=sin²(x), ST(2)=sin(x), ST(3)=zzz
fsubr           ;ST(0)=1-sin²(x)=cos²(x), ST(1)=sin(x), ST(2)=zzz
fsqrt           ;ST(0)=cos(x), ST(1)=sin(x), ST(2)=zzz
fpatan          ;ST(0)=arcsin[sin(x)]=x (in radians), ST(1)=zzz
```

Since the derived cos(x) value would always be positive, the resulting angle would be in the -90° to 0° to +90° ranges. The sign of the derived cos(x) value would need to be changed to get an angle in the +90° to 180° to -90° ranges.

The arccosine can be computed in a similar manner. The only difference would be that the cos(x) value and the derived sin(x) value would have to be exchanged before the FPATAN instruction. If the positive sign of the sin(x) value is retained, the resulting arccosine value would be in the 0° to +90° to +180° ranges. The sign of the derived sin(x) value would need to be changed to get an angle in the -0° to -90° to -180° ranges.

RETURN TO
SIMPLY FPU
CONTENTS